Lab 7: Fuzzing for Input Validation Bugs

Software Engineering | Repository: L6_L7_18_atm_withdrawal_SE (ATM Withdrawal)

NAME: Devika N	SRN: PES1UG24AM080	SECTION: B

OBJECTIVE

To use fuzz testing with the Hypothesis library to find input validation bugs in the three functions of atm_withdrawal.py (notes_needed, mask_card and average_withdrawal), fix them, and confirm the fix by re-running the fuzz tests.

RUNNING THE BUGGY CODE

The original atm_withdrawal.py was run first. With valid input it works correctly. With an invalid amount such as "abc" it crashes.

```
C:\Users\Narayan Kutty\Desktop\L6_L7_18_atm_withdrawal_SE\Lab 7>  python buggy_atm_withdrawal.py
==== Running notes_needed ====
Enter an amount (e.g. 2,500): 2500
Number of 100 Notes: 25

==== Running mask_card ====
Enter a card number (e.g. 1234 5678 9012 3456): 1234 5678 9012 3456
Masked Card: XXXX XXXX XXXX 3456

==== Running average_withdrawal ====
Enter withdrawals as CSV (e.g. 500,1000,2000): 500,1000,2000
Average Withdrawal: 1166

C:\Users\Narayan Kutty\Desktop\L6_L7_18_atm_withdrawal_SE\Lab 7>  python buggy_atm_withdrawal.py
==== Running notes_needed ====
Enter an amount (e.g. 2,500): abc
Traceback (most recent call last):
  File "C:\Users\Narayan Kutty\Desktop\L6_L7_18_atm_withdrawal_SE\Lab 7\buggy_atm_withdrawal.py", line 29, in <module>
    main()
  File "C:\Users\Narayan Kutty\Desktop\L6_L7_18_atm_withdrawal_SE\Lab 7\buggy_atm_withdrawal.py", line 15, in main
    result = notes_needed(user_input)
             ^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\Narayan Kutty\Desktop\L6_L7_18_atm_withdrawal_SE\Lab 7\buggy_atm_withdrawal.py", line 2, in notes_needed
    return int(amount.replace(',', '')) // 100
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'abc'

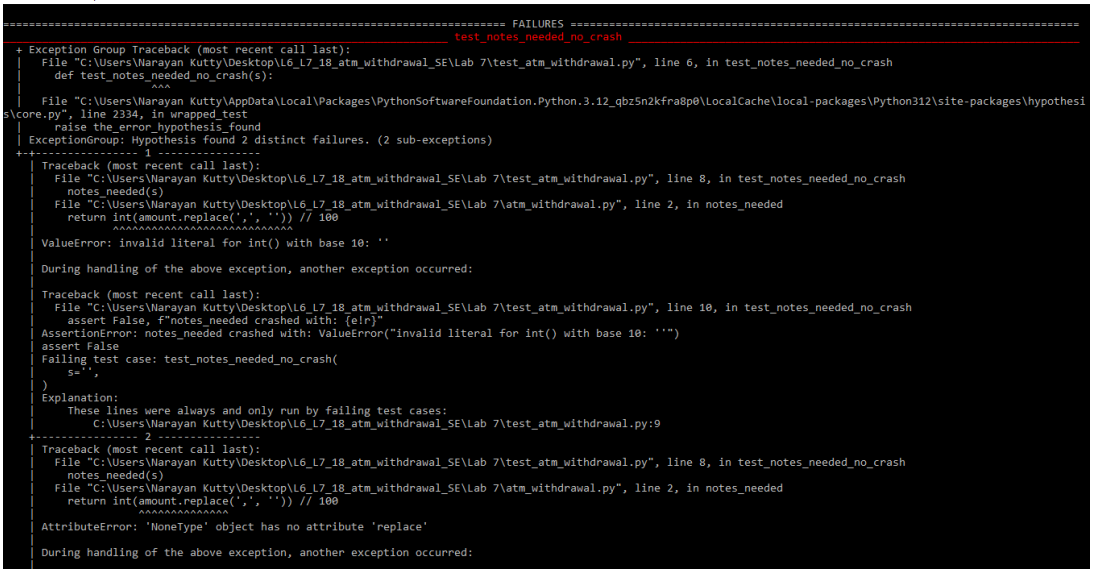
C:\Users\Narayan Kutty\Desktop\L6_L7_18_atm_withdrawal_SE\Lab 7>
```

Entering "abc" crashes with ValueError.

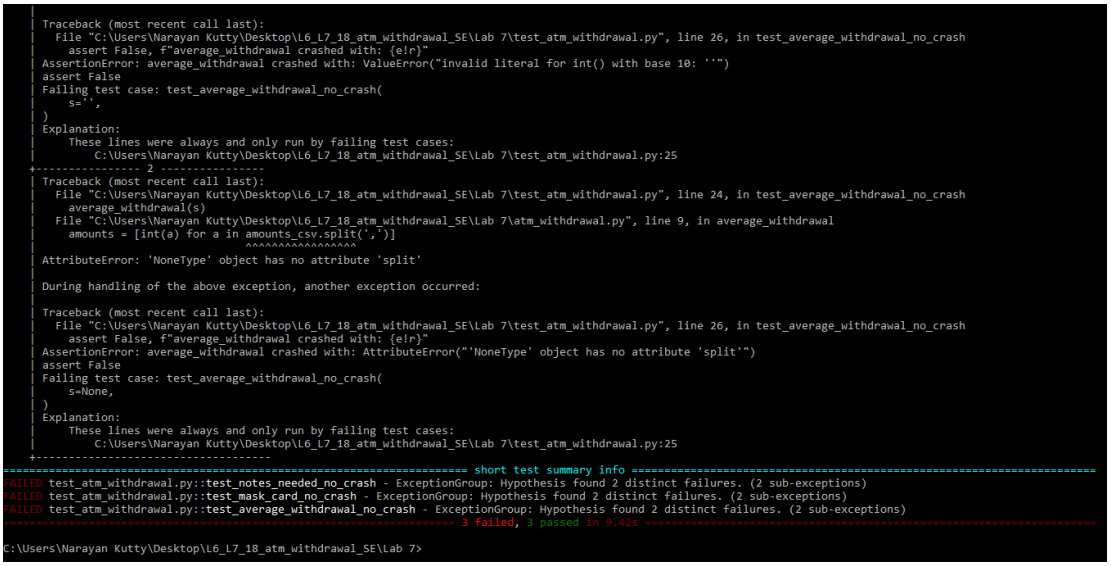
notes_needed passes user text straight to int() with no validation, so any non-numeric input causes error.

Fuzz tests on the buggy code

test_atm_withdrawal.py was completed with three Hypothesis tests. Each calls one function with st.text() | st.none() and fails if any exception is raised. Three extra tests check that valid inputs still give correct answers.



Hypothesis finds two distinct failures in test_notes_needed_no_crash: s='' (ValueError) and s=None (AttributeError).



Test summary on the buggy code: 3 failed, 3 passed.

Bugs Found

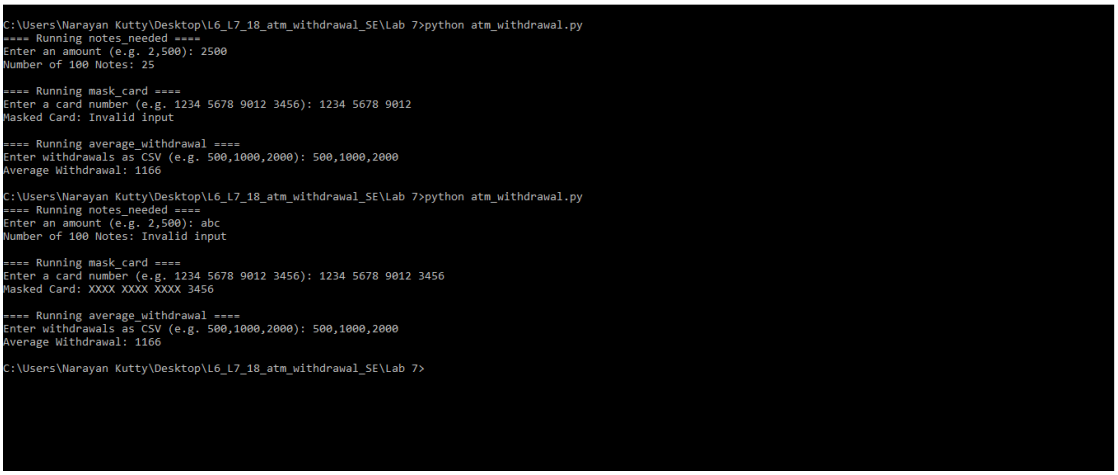
Function	Failing input	Error in buggy code	Fix applied
notes_needed	Empty string	ValueError: invalid literal for int()	Check input is a string of digits; otherwise return None
notes_needed	None	AttributeError: 'NoneType' has no attribute 'replace'	isinstance(str) check first
mask_card	Empty string	IndexError: list index out of range	Require exactly 4 groups of 4 digits
mask_card	None	AttributeError: 'NoneType' has no attribute 'split'	isinstance(str) check first
average_withdrawal	Empty string	ValueError: invalid literal for int()	Validate every comma-separated part is digits

Function	Failing input	Error in buggy code	Fix applied
average_withdrawal	None	AttributeError: 'NoneType' has no attribute 'split'	isinstance(str) check first

Fix Application

Each function was hardened so that invalid input never raises an exception. Every function now:

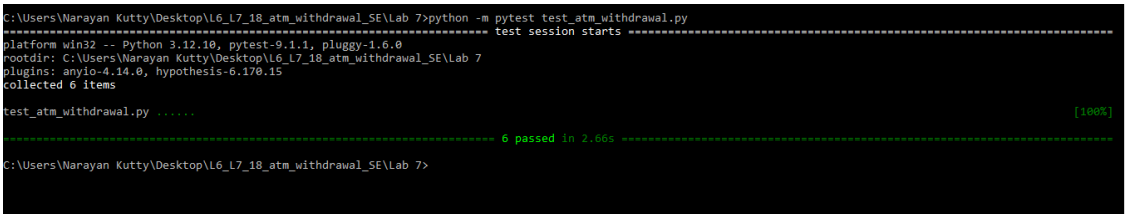
- checks the input is a string (isinstance) before using it;
- validates the format (digits only; exactly 4 groups of 4 digits for a card number);
- returns None for invalid input, and main() prints "Invalid input" in that case.



Fixed script. An incomplete card number gives "Invalid input". “abc" gives "Invalid input" instead of crashing.

The same inputs are now handled safely, and the output for valid inputs is unchanged.

Re-run the fuzz tests on the fixed code



After the fix, all 6 tests pass.

Hypothesis generated many random inputs, including empty strings, None and arbitrary Unicode text, and none caused a crash. The correctness tests also still pass, so the fix did not change behaviour for valid input.

Conclusion

Fuzz testing found input validation bugs that normal example-based testing would likely have missed: empty strings and None crashed all three functions. After adding defensive input checks, the same fuzz tests pass, which shows the code is now more robust. Property-based testing is useful because it tries inputs the developer did not think of.